

# Tracked Training Pipeline

## Product Requirements Document (PRD)

Track: Multimodal AI -- Deep Learning & Generative Architectures  
Stack: Python . PyTorch . FastAPI . React . Tailwind CSS . Persistent Storage  
Duration: 12 Weeks | Team Size: 1 (Solo Developer) | Level: Advanced

Grounding Paper: PyTorch: An Imperative Style,  
High-Performance Deep Learning Library -- NeurIPS 2019  
Dataset: CIFAR-10 (60,000 images x 10 classes)

## 2. Executive Summary & Product Overview

### 2.1 Core Product Direction & Positioning

"TrainTrace is a focused ML experimentation and lifecycle platform that enables users to design, execute, compare, reproduce, evaluate, and make evidence-based decisions about machine-learning experiments and model candidates."

TrainTrace is designed specifically for academic, research, and On-the-Job Training (OJT) environments. It is not positioned as an enterprise-grade replacement for industrial MLOps platforms such as MLflow or Weights & Biases. Instead, TrainTrace focuses on the core academic theme: REPRODUCIBLE DEEP LEARNING + MLOps. It provides a self-contained, lightweight implementation that emphasizes experiment reasoning, hypothesis validation, model candidate selection evidence, and end-to-end provenance.

### 2.2 Distinguishing Model Comparison from Decision Traceability

The PRD explicitly distinguishes two complementary capabilities:

- \* Model Comparison ("What were the results?"): Evaluates metrics (loss, accuracy, latency, model size) side-by-side to identify which approach performed better under tested parameters.
- \* Decision Traceability ("Why did we choose this approach and reject others?"): Preserves the human reasoning, motivation, trade-off evaluations, limitations, and historical sequence that led from initial hypotheses through alternative trials to the final selected model.

*Core Platform Principle: TrainTrace does not automatically decide which model is best, nor does it merely record what was tried and what worked. It preserves why approaches were investigated, why alternative approaches were rejected or superseded, what evidence supported those decisions, and why the final approach was selected so that any reviewer or future collaborator can understand how the decision was reached.*

## 2.3 Preservation of Mandatory OJT Requirements

While expanding the product to cover model candidate lifecycles, registries, and decision tracking, TrainTrace fully preserves all original OJT technical mandates:

- \* PyTorch Training Pipeline: Modular training loop incorporating explicit forward passes, loss computation, and autograd backpropagation.
- \* DataLoaders & Preprocessing: Standardized PyTorch DataLoaders with customizable batching, shuffling, and image transformations.
- \* Checkpoint State Management: Periodic and completion-based state serialization (.pt files).
- \* Experiment Tracking: Local metric logging (loss, accuracy, epoch duration) and optional cloud synchronization.
- \* Reproducibility Analysis: Deterministic fixed-seed initialization protocols across random number generators (RNGs).
- \* CIFAR-10 Benchmark: Required baseline image-classification workload.
- \* Full-Stack Application: Responsive React frontend single-page application (SPA) backed by a FastAPI REST API engine.

## 2.4 System Architecture Overview

The platform comprises an interactive React SPA dashboard, a FastAPI application backend, a PyTorch execution engine, an experiment and lineage persistence layer, a model candidate decision engine, and an integrated FastAPI model serving interface.

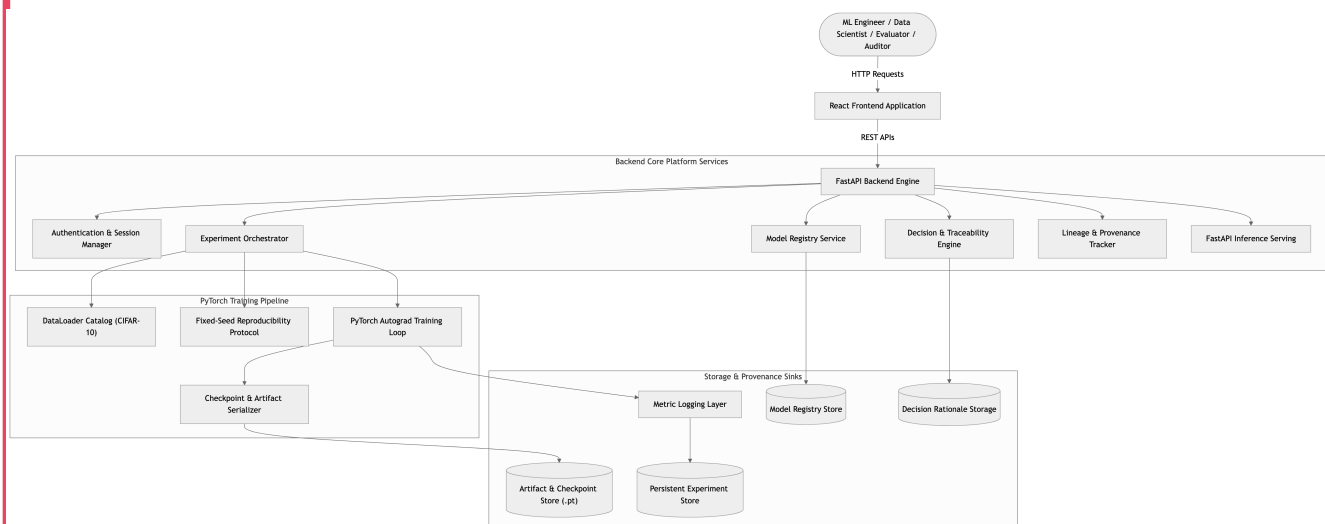


Figure 1 -- TrainTrace ML Experimentation & Lifecycle Platform Architecture. Schematic illustrating component relationships between the React SPA, FastAPI Backend Services, PyTorch Training Engine, Model Registry, Decision Engine, Lineage Tracker, and Storage Layers.

*Section takeaway: TrainTrace balances formal OJT PyTorch and full-stack requirements with expanded MLOps capabilities, treating ML experimentation as a structured, traceable decision process.*

### 3. Problem Statement

---

Machine learning development in research and early-stage engineering frequently suffers from structural operational deficiencies:

1. **Untracked Trial Runs & Lost Context:** Engineers execute dozens of training runs via ad-hoc terminal scripts. Parameters, loss curves, and architectural variations are stored in unorganized text logs or local spreadsheets, losing the rationale behind experiment choices.
2. **Lack of Decision Traceability:** Standard experiment tracking tools record raw metrics, but fail to record why a particular model architecture was selected, why alternative approaches were rejected, or what trade-offs governed the decision. When team members leave or projects pause, the reasoning chain is lost.
3. **Non-Determinism & Unverifiable Results:** Deep learning scripts often introduce silent non-determinism via unseeded RNGs, arbitrary data shuffling, and unpinned worker processes. Consequently, researchers cannot verify whether accuracy gains stem from hyperparameter improvements or random seed variation.
4. **Inaccessible Model Checkpoints & State Disconnection:** Model weights saved during training are frequently disconnected from their originating configuration, dataset split, preprocessing pipeline, and commit version, leading to unidentifiable .pt files.
5. **Tooling Complexity:** Existing enterprise tools require complex cloud infrastructure, external server setups, and paid subscriptions, creating unnecessary friction for local academic and OJT projects.

*Section takeaway: TrainTrace resolves experiment fragmentation by recording not only training metrics, but also decision history, model lineage, and empirical reproducibility evidence.*

---

### 4. Product Vision

To provide an accessible, self-contained ML experimentation and lifecycle platform that demonstrates core PyTorch deep learning principles while enabling researchers and engineers to design structured investigations, execute seed-controlled runs, evaluate trade-offs, capture lineage provenance, register model versions, and defend model selection decisions with empirical evidence and explicit reasoning chains.

---

### 5. Product Goals and Measurable Success Criteria

#### 5.1 Product Goals

1. **Establish First-Class Decision Traceability:** Provide a first-class capability to capture, store, and display the reasoning chain (Objective -> Approach -> Evidence -> Limitations -> Decision -> Alternative -> Next Investigation) across all experimentation attempts.
2. **Establish Deterministic & Statistical Reproducibility:** Support dual-mode reproducibility protocols distinguishing (A) exact repeatability under matching conditions and (B) sensitivity analysis across controlled random seeds.
3. **Deliver End-to-End Model Lineage:** Enable complete provenance tracing from any registered model version back to its originating run, experiment investigation, dataset split, Git commit, and environment configuration.

4. Maintain Operational Simplicity: Deliver a full-stack web application executable in a local environment within a 12-week solo development scope.

## 5.2 Measurable Product Success Criteria

| Success Dimension          | Measurable Outcome Target                                                                                                                        | Verification Method                                      |
|----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------|
| Decision Traceability      | 100% of candidate decision states (Selected, Rejected, Superseded) require recorded justification, alternative analysis, and linked run metrics. | Automated validation in Decision Engine API              |
| Reasoning Chain Coverage   | 100% of selected model versions can display an unbroken investigation history showing motivating limitations.                                    | Audit view inspection in UI / API                        |
| Model Provenance           | 100% of registered model versions can be traced back to exact run configuration, dataset version, Git commit, and environment metadata.          | Lineage graph inspection via UX / API                    |
| Repeatability Verification | Repeated execution of a run under identical seed, config, and environment yields identical metric trajectories and test accuracy.                | Automated test suite evaluation across N=2 matching runs |
| Sensitivity Analysis       | Automated calculation of Mean, Standard Deviation, Min, Max, and Spread across N=5 controlled seed executions.                                   | Variance Report generation engine                        |
| Inference Connection       | Selected and registered model versions can be loaded into the FastAPI inference service for real-time sample predictions.                        | End-to-end manual and automated API testing              |

Section takeaway: Success is defined by rigorous decision tracking, verifiable reproducibility, full model lineage, and

*seamless end-to-end user workflows.*

## 6. Stakeholders

TrainTrace serves distinct roles across academic, technical, and operational evaluation contexts.

### 6.1 Product Users (Target Stakeholders)

1. ML Engineer: Configures training pipelines, executes runs, monitors real-time training progress, inspects state checkpoints, and links new approaches to the limitations that motivated them.
2. Data Scientist / AI Researcher: Formulates hypotheses, designs alternative experiment approaches, compares candidate models, conducts reproducibility analyses, and records why approaches were rejected.
3. MLOps Engineer: Manages model candidate lifecycles, tracks model versions in the registry, inspects environment/code lineage, and manages promotion to inference.
4. ML Lead / Team Lead: Reviews experiment progress across team investigations, inspects trade-off analysis tables, and evaluates decision rationale reports.
5. Model Evaluator / QA: Validates model quality, verifies test set evaluation metrics, checks seed sensitivity reports, and audits reproducibility claims.
6. Product / Technical Decision Maker: Seeks understandable trade-off comparisons (accuracy vs inference speed vs model size) and explicit explanations of why alternative approaches were not chosen.
7. Reviewer / Auditor: Examines end-to-end provenance, dataset versions, hyperparameter history, supporting evidence, and recorded decision rationales to evaluate and defend model selection decisions.

### 6.2 Project & Academic Stakeholders

1. Student / Developer (Author): Designs, implements, tests, and defends the TrainTrace platform as part of a 12-week OJT curriculum.
2. Project Mentor / Evaluator: Assesses technical execution, adherence to OJT requirements, PyTorch pipeline implementation, and document rigor.
3. Academic Institution: Sets curriculum standards, evaluation criteria, and project defense guidelines.

*Section takeaway: TrainTrace accommodates diverse perspectives -- from low-level PyTorch engineering to high-level model decision auditing -- through unified, role-relevant UI views.*

## 7. Target Users / Personas

| Persona Title                     | Primary Goal                                                                                              | Key Pain Points                                                                                                   | Technical Profile & UI Needs                                                                                                          |
|-----------------------------------|-----------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| Alex -- ML Engineer               | Efficiently launch, monitor, and tune PyTorch training runs; link runs to motivating limitations.         | Terminal clutter, lost .pt files, manual spreadsheet tracking, losing context on why past runs failed.            | High Python/PyTorch proficiency. Needs fast launch forms, real-time charts, downloadable checkpoints, and investigation links.        |
| Dr. Elena -- AI Researcher        | Test hypotheses, compare alternative architectures, verify reproducibility, and record rejection reasons. | Non-deterministic training scripts, lack of controlled variance analysis, forgotten reasons for abandoning ideas. | Deep ML theory expertise. Needs structured experiment hierarchies, seed controls, variance reporting, and decision rationale capture. |
| Marcus -- MLOps Engineer          | Maintain model versions, track code/dataset lineage, and deploy selected candidates.                      | Disconnected model files, missing environment metadata, untracked code versions.                                  | Strong infrastructure/DevOps skills. Needs model registry views, lineage graphs, and inference endpoint management.                   |
| Sarah -- ML Lead / Decision Maker | Evaluate trade-offs and understand why selected approaches were preferred over alternatives.              | Raw metric charts that lack business context, decision trade-offs, or explicit rejection rationales.              | Strategic technical leadership. Needs high-level decision reports, comparison tables, and alternative analysis views.                 |

Section takeaway: Personas guide UI layout design, ensuring low-level metric visualization and high-level decision tracking coexist harmoniously.

## 8. Key Product Concepts

To support an evidence-based ML lifecycle, TrainTrace defines nine first-class product concepts:

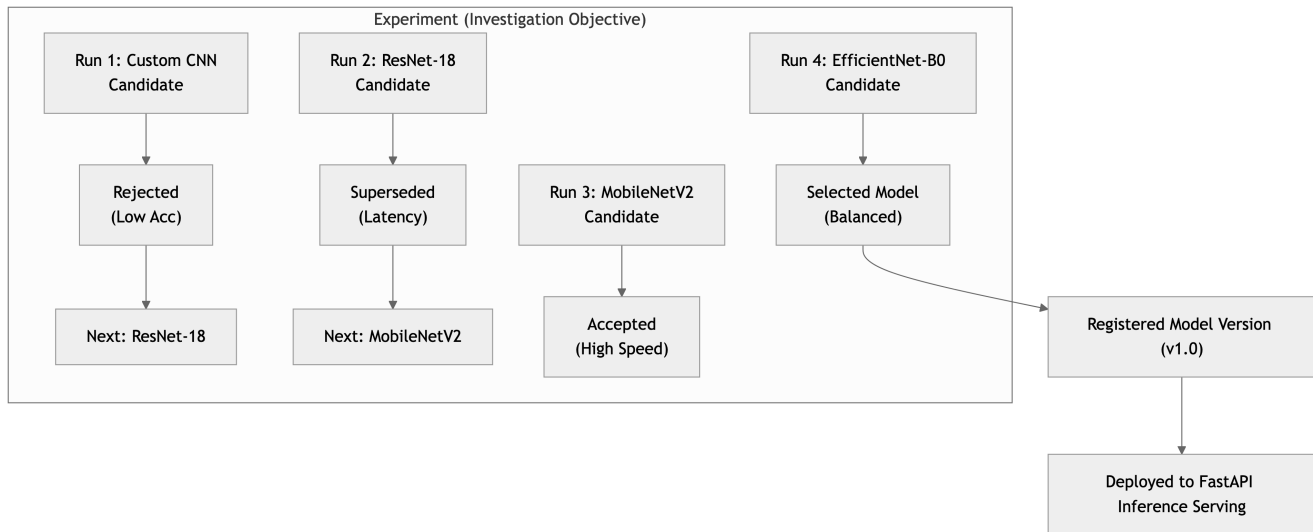


Figure 6 -- Key Product Concepts Hierarchy. Diagram mapping an investigation objective to candidate runs, decision evaluations (Rejected, Superseded, Accepted, Selected), model registration, and deployment.

## 8.1 Conceptual Definitions

- \* **Experiment (Investigation):** A defined ML investigation or objective containing a hypothesis, target metrics, related training runs, evaluation evidence, decision records, and final conclusions.
- \* **Run (Execution):** One actual execution of a specific training configuration, dataset split, hyperparameter set, and seed initialization.
- \* **Model Candidate:** A trained neural network artifact resulting from one or more runs, evaluated for suitability against project objectives.
- \* **Decision:** The explicit evaluation outcome assigned to a Model Candidate (Candidate, Accepted, Rejected, Superseded, or Selected), accompanied by recorded rationale and supporting metric evidence.
- \* **Experiment Investigation & Decision Traceability (First-Class Capability):** The product capability that captures, preserves, and displays the complete reasoning chain and historical relationships between experiment approaches, observed limitations, rejection rationales, and candidate selection decisions.
- \* **Model Version:** A specific, registered version of a selected model candidate in the Model Registry (e.g., ResNet18-CIFAR10:v1.0), locked to source lineage.
- \* **Artifact:** A useful generated output associated with a run or model, including state checkpoints (.pt), training curves, evaluation reports, confusion matrices, and prediction logs.
- \* **Dataset:** A managed dataset entity detailing dataset identity (e.g., CIFAR-10), dataset version, train/validation/test split configuration, and preprocessing/augmentation transforms.
- \* **Lineage / Provenance:** The complete, traceable graph linking a registered model version to its originating run, experiment investigation, dataset configuration, Git commit hash, and runtime environment.

## 8.2 The ML Experimentation Reasoning Chain

TrainTrace preserves the following explicit 11-step reasoning chain across ML experimentation:

Reasoning Chain Flow:

Objective / Problem -> Approach -> Run(s) -> Evidence / Results -> Limitations / Observed Issues -> Decision -> Decision Reason -> Alternative Considered -> Why Not Alternative -> Next Investigation -> Final Selected Approach

### 8.3 Capturing 11 Core Decision Attributes per Approach/Candidate

For each meaningful experiment approach or candidate, TrainTrace captures eleven standardized product attributes:

1. Objective: What problem or goal is being investigated?
2. Approach: What model architecture, dataset split, hyperparameter profile, or training technique was tried?
3. Evidence: What measurable evaluation results support the evaluation? (e.g., test accuracy, loss, inference latency, model size, reproducibility spread).
4. Strengths: What did the approach do well? (e.g., fast convergence, small parameter count).
5. Limitations / Observed Issues: What trade-off, weakness, or bottleneck was discovered? (e.g., insufficient accuracy, high latency, memory usage).
6. Decision Status: Candidate, Accepted, Rejected, Superseded, or Selected.
7. Decision Reason: Why was the approach assigned its decision status?
8. Alternative Considered: What other approach could have been used?
9. Why Not the Alternative: Why was the chosen approach preferable under the project's evaluation criteria and constraints?
10. Next Investigation: If an approach was rejected or found insufficient, what was investigated next and why?
11. Relationship Between Investigations: How one investigation links to another (parent/ancestor investigation pointers).

### 8.4 Concrete Illustrative Example (Investigation Reasoning Chain)

The following concrete example illustrates how TrainTrace captures an experimentation investigation chain. (Note: Metric values and decision choices are purely illustrative examples of product behavior, not mandatory project thresholds).

- \* Overall Objective: Improve image-classification performance while keeping the solution practical for deployment.
- \* Approach 1 -- Custom CNN (Baseline):
  - \* Result / Evidence: Baseline performance (~74.2% test accuracy).
  - \* Limitation: Insufficient classification accuracy on complex image features.
  - \* Decision Status: Rejected.
  - \* Decision Reason: Did not satisfy the accuracy evaluation objective.
  - \* Next Investigation: ResNet-18.
  - \* Why Next: Investigate whether a deeper established residual architecture addresses feature representation limitations.
- \* Approach 2 -- ResNet-18:
  - \* Result / Evidence: Significantly improved performance (~88.5% test accuracy).
  - \* Limitation: Higher computational complexity and larger model size.
  - \* Decision Status: Superseded.
  - \* Decision Reason: Superseded after candidate evaluation revealed an unfavorable deployment latency trade-off compared to lighter architectures.
  - \* Next Investigation: MobileNetV2.
  - \* Why Next: Investigate whether comparable performance can be achieved with a lighter depthwise-separable architecture.
- \* Approach 3 -- MobileNetV2:
  - \* Result / Evidence: Extremely fast inference (~82.1% test accuracy, lightweight size).
  - \* Limitation: Accuracy drop on fine-grained test samples.
  - \* Decision Status: Accepted (Candidate) / Rejected (Final).

- \* Decision Reason: Efficiency improvement did not justify the 6.4% performance reduction under target criteria.
- \* Next Investigation: EfficientNet-B0.
- \* Why Next: Investigate compound scaling architecture for optimal balance.
- \* Approach 4 -- EfficientNet-B0:
  - \* Result / Evidence: Strongest overall result (~91.4% test accuracy, balanced latency and parameter size).
  - \* Limitation: Slightly longer training time per epoch.
  - \* Decision Status: Selected.
  - \* Decision Reason: Recorded evidence supports the preferred overall trade-off between accuracy, model size, and inference speed.

*Section takeaway: Concepts clearly decouple investigations (Experiments) from single executions (Runs), establishing a formal model candidate lifecycle, registry, and an explicit 11-step decision reasoning chain.*

## 9. Product Workflow & User Journey

The TrainTrace user journey spans four primary phases: Investigation Setup, Execution & Tracking, Evaluation & Reproducibility, and Candidate Decision & Lifecycle.

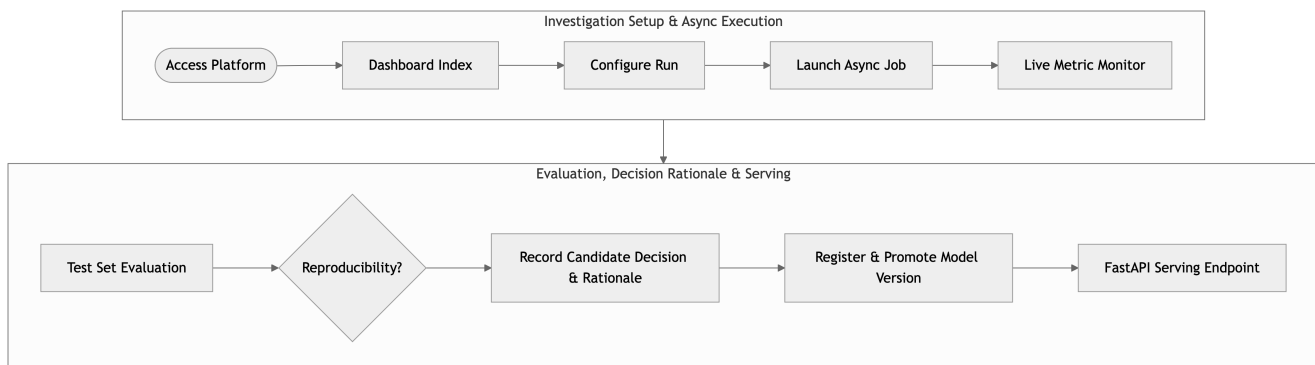


Figure 2 -- Stakeholder User Journey & Decision Flow. Flowchart depicting user progression from defining objectives through launching runs, evaluating metrics, executing reproducibility audits, recording decisions, and deploying models to inference.

### 9.1 Investigation Branching & Decision Lineage

Investigations naturally branch as initial hypotheses yield new insights. TrainTrace makes this investigation history transparent by linking successor investigations directly to motivating limitations.

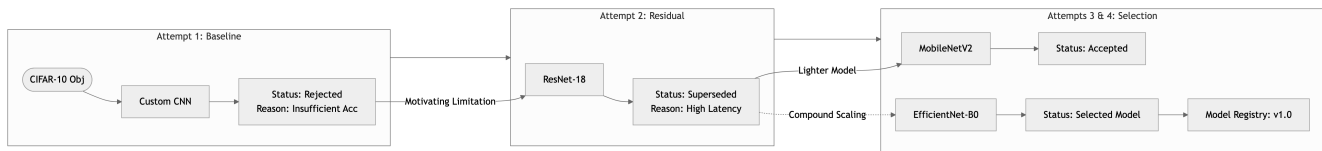


Figure 3 -- Investigation Lineage & Experiment Branching Workflow. Diagram showing how an initial objective branches across alternative approaches (Custom CNN -> ResNet-18 -> MobileNetV2 -> EfficientNet-B0), recording rejection reasons, successor links, and final model promotion.

Section takeaway: The workflow connects initial hypothesis formation directly to candidate evaluation, decision recording, decision rationale preservation, and model promotion.

## 10. Use Cases

### 10.1 UC-01: Configure and Launch Training Run

- \* Primary Actor: ML Engineer
- \* Precondition: User is authenticated and on the "New Run" page within an Experiment.
- \* Main Flow:
  1. User selects target dataset (e.g., CIFAR-10) and model architecture (e.g., ResNet-18).
  2. User configures hyperparameters: learning rate (e.g., 0.001), batch size (128), epochs (30), optimizer (AdamW), and random seed (42).
  3. User links run to motivating investigation limitation (e.g., "Address Custom CNN accuracy limitation").
  4. User clicks Launch Training.
  5. System validates parameters, initializes job asynchronously, sets state to RUNNING, and redirects user to Live Run Detail view.

### 10.2 UC-02: Monitor Live Training Trajectory

- \* Primary Actor: ML Engineer / AI Researcher
- \* Precondition: Training run is in RUNNING state.
- \* Main Flow:
  1. User views Live Run Detail page.
  2. System streams per-epoch metrics (train loss, val loss, train accuracy, val accuracy, epoch duration).
  3. Interactive line charts render updated curves in real time.
  4. Upon completion, system triggers automatic holdout test set evaluation and updates state to COMPLETED.

### 10.3 UC-03: Compare Candidate Models & Record Decision Rationale

- \* Primary Actor: ML Lead / AI Researcher
- \* Precondition: Multiple runs under an Experiment have reached COMPLETED state.
- \* Main Flow:
  1. User navigates to Experiment Comparison view and selects candidate runs.
  2. System renders overlaid metric charts, parameter summary tables, and resource consumption comparisons (training time, model size, latency).

3. User evaluates trade-offs (e.g., MobileNet high speed vs EfficientNet high accuracy).
4. User selects EfficientNet candidate and clicks Record Decision.
5. User marks EfficientNet as Selected (recording rationale, strengths, limitations, and why alternatives were not chosen) and marks ResNet-18 as Superseded.
6. System persists decision attributes and updates the Experiment Decision Report.

#### 10.4 UC-04: Execute Reproducibility Analysis

- \* Primary Actor: AI Researcher / Model Evaluator
- \* Precondition: Target run configuration exists.
- \* Main Flow:
  1. User opens Reproducibility tab on a completed run.
  2. User selects protocol mode: Deterministic Repeatability (matching seed) or Sensitivity Analysis (N=5 controlled seeds).
  3. System executes protocol, collects run results, and calculates Mean, Standard Deviation, Min, Max, and Spread.
  4. System displays statistical Variance Report in UI and attaches report artifact to the Experiment.

#### 10.5 UC-05: Register Model & Deploy to FastAPI Inference

- \* Primary Actor: MLOps Engineer
- \* Precondition: Model candidate is marked Selected.
- \* Main Flow:
  1. User clicks Register Model Version.
  2. System creates registered version ResNet18-CIFAR10:v1.0, locking model checkpoint, dataset split, Git commit, environment metadata, and decision rationale.
  3. User clicks Promote for Inference.
  4. System loads model checkpoint into FastAPI inference endpoint.
  5. User uploads or selects a test image in Inference UI; system returns predicted class label, confidence scores, and model version metadata.

#### 10.6 UC-06: Trace Investigation History & Decision Rationale

- \* Primary Actor: Reviewer / Auditor / Technical Decision Maker
- \* Precondition: Registered model version exists with documented investigation history.
- \* Main Flow:
  1. User navigates to Model Registry or Decision Report view.
  2. User clicks View Investigation History.
  3. System displays interactive reasoning chain from initial objective through rejected/superseded approaches to final selected model version.
  4. User inspects rejection reasons, strengths, limitations, and supporting metric evidence for each intermediate approach.

*Section takeaway: Use cases cover the entire lifecycle -- from single-run launching to multi-run comparison, decision recording, decision history tracing, and real-time inference serving.*

## 11. Product Scope

---

### 11.1 Controlled Dataset Scope

TrainTrace avoids arbitrary dataset uploads to maintain platform stability and benchmarking consistency. It defines a controlled catalog of compatible image-classification datasets:

- \* Mandatory Benchmark Dataset (Required OJT Workload): CIFAR-10 (60,000 32x32 color images, 10 classes).
- \* Planned Supported Dataset Catalog: CIFAR-100, MNIST, Fashion-MNIST, SVHN.
- \* Dataset Extensibility: Pluggable dataset loader interface supporting standard torchvision formats.

### 11.2 Controlled Model Architecture Scope

TrainTrace supports structured, compatible image-classification model families:

- \* Baseline Custom CNN: Lightweight 3-layer convolutional neural network for rapid baseline training.
- \* ResNet Family: ResNet-18 (residual learning baseline for robust feature extraction).
- \* MobileNet Family: MobileNetV2 (depthwise separable convolutions for efficient inference).
- \* EfficientNet Family: EfficientNet-B0 (compound scaling for high accuracy/parameter efficiency).

### 11.3 Scope Prioritization (MoSCoW Summary)

- \* Must Have (12-Week Core): CIFAR-10 dataset, PyTorch training loop (autograd, DataLoaders, checkpoints), seed reproducibility protocols, local metric logging, experiment/run hierarchy, Experiment Investigation & Decision Traceability engine (11 attributes, reasoning chain, status recording, rejection reasons), model candidate status, model registry versioning, trade-off comparison view, FastAPI inference serving, React dashboard.
- \* Should Have: Planned dataset catalog (CIFAR-100, MNIST), code Git commit tracking, environment metadata collection, downloadable .pt checkpoints, decision report generation, W&B optional sync.
- \* Could Have: MobileNet/EfficientNet model integrations, confusion matrix visualizer, learning rate scheduler preview.
- \* Won't Have (MVP): Distributed multi-GPU DDP, arbitrary user code upload, automated hyperparameter optimization (Bayesian/Grid), production cloud deployment.

*Section takeaway: Scope is strictly bounded around vision classification datasets and architectures, ensuring solo 12-week project feasibility.*

---

## 12. Functional Requirements

---

### 12.1 Experiment & Investigation Management

- \* FR-EXP-01: The system shall provide an Experiment Management interface enabling users to create, view, edit, and archive ML investigations.
- \* FR-EXP-02: Each Experiment shall store an explicit Objective, Hypothesis, Target Metrics, Associated Runs, Candidate Decisions, Decision Rationales, and Summary Conclusion.

- \* FR-EXP-03: The system shall support parent-child investigation linking to visualize investigation branching history and successor links.

## 12.2 Dataset Management

- \* FR-DAT-01: The system shall maintain a Dataset Catalog featuring CIFAR-10 (Required Benchmark), CIFAR-100, MNIST, Fashion-MNIST, and SVHN.
- \* FR-DAT-02: The system shall record dataset configuration metadata for each run, including dataset identity, dataset version, train/validation/test split ratios, batch size, and image preprocessing/augmentation transforms.

## 12.3 Model & Approach Management

- \* FR-MDL-01: The system shall provide supported model families (Custom CNN, ResNet-18, MobileNetV2, EfficientNet-B0) compatible with the dataset catalog.
- \* FR-MDL-02: The system shall validate model architecture compatibility with selected dataset dimensions prior to initiating training.

## 12.4 Training Execution

- \* FR-EXE-01: The system shall execute PyTorch training loops incorporating explicit forward passes, loss calculation, autograd backward passes, and optimizer stepping.
- \* FR-EXE-02: Training execution shall run asynchronously in the background, keeping the web interface interactive during training.
- \* FR-EXE-03 (Single Active Job Handling): If a user launches a training run while another run is active, the system shall notify the user, offering to queue the request or reject it with an explanatory message.
- \* FR-EXE-04 (Stop / Cancellation Handling): Users shall be able to manually halt a running job. The system shall retain all metrics and checkpoints generated prior to cancellation, marking state as STOPPED.

## 12.5 Experiment Tracking & Metric Logging

- \* FR-TRK-01: The system shall log per-epoch metrics: Training Loss, Validation Loss, Training Accuracy, Validation Accuracy, Learning Rate, and Epoch Duration.
- \* FR-TRK-02: All metrics shall be persisted locally to ensure tracking operates reliably without internet connectivity.
- \* FR-TRK-03: When optional W&B credentials are configured, the system shall synchronize logged epoch metrics to the user's W&B workspace.

## 12.6 Evaluation

- \* FR-EVL-01: Upon training loop completion, the system shall execute an automated evaluation pass over the holdout test dataset, recording final Test Loss and Test Accuracy.
- \* FR-EVL-02: The system shall compute secondary evaluation metrics (Precision, Recall, F1-Score) and generate confusion matrix data for candidate models.

## 12.7 Reproducibility Engine (Repeatability vs Sensitivity)

- \* FR-REP-01 (Deterministic Seeding): The system shall enforce seed initialization across PyTorch (torch.manual\_seed), NumPy (np.random.seed), Python random, and DataLoader worker processes (worker\_init\_fn).

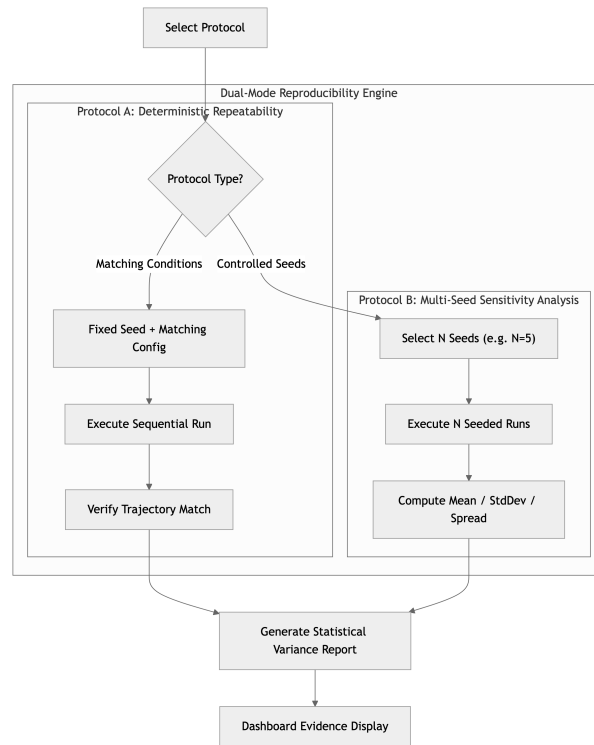


Figure 4 -- Reproducibility Protocol & Evaluation Workflow. Flowchart illustrating Protocol A (Deterministic Repeatability) and Protocol B (Multi-Seed Sensitivity Analysis).

- \* FR-REP-02 (Protocol A -- Matching Condition Repeatability): The system shall allow re-running an experiment under identical configuration, seed, and environment to verify metric trajectory match.
- \* FR-REP-03 (Protocol B -- Seed Sensitivity Analysis): The system shall support automated execution across N controlled seeds (default N=5) to compute Mean Accuracy, Standard Deviation, Min, Max, and Variation Spread.

## 12.8 Model Comparison & Trade-Off Analysis

- \* FR-CMP-01: The system shall support side-by-side comparison of up to 4 runs within an Experiment.
- \* FR-CMP-02: Comparison views shall display overlaid loss/accuracy trajectory charts, hyperparameter side-by-side tables, final test accuracies, training duration, and estimated model parameter size.

## 12.9 Experiment Investigation & Decision Traceability (Core Capability)

- \* FR-DEC-01 (Decision Rationale Recording): The system shall allow users to record a formal textual rationale and metric evidence when assigning a decision status to an approach or model candidate.
- \* FR-DEC-02 (Investigation Pointers): The system shall allow users to link a new approach or investigation to the specific limitation, observed problem, or hypothesis that motivated it.
- \* FR-DEC-03 (Evidence Preservation): The system shall preserve all quantitative evaluation metrics (accuracy, loss, latency, model size, reproducibility spread) associated with a candidate decision.
- \* FR-DEC-04 (Candidate Decision Statuses): The system shall support assigning candidate decision statuses: Candidate, Accepted, Rejected, Superseded, or Selected.
- \* FR-DEC-05 (Alternative Analysis Capture): The system shall allow users to record why alternative approaches were not chosen under project evaluation criteria.
- \* FR-DEC-06 (Investigation History View): The system shall provide a visual investigation history view displaying the complete reasoning chain leading to a selected model version.

## 12.10 Artifact Management

- \* FR-ART-01: The system shall automatically save model state checkpoints (.pt files containing model state\_dict and optimizer state\_dict) at periodic epoch intervals and upon run completion.
- \* FR-ART-02: The system shall manage run artifacts, including saved weights (.pt), loss/accuracy CSV logs, evaluation report PDFs, and confusion matrix image renders.
- \* FR-ART-03: Users shall be able to list, inspect, and download all artifacts directly from the web dashboard.

## 12.11 Model Registry & Versioning

- \* FR-REG-01: The system shall feature a Model Registry where selected candidate models can be formally registered with semver tags (e.g., ResNet18-CIFAR10:v1.0).
- \* FR-REG-02: Registered model versions shall lock and preserve pointers to originating run ID, experiment ID, checkpoint artifact, dataset configuration, evaluation report, decision record, Git commit, and environment snapshot.

## 12.12 Lineage & Provenance Tracking

- \* FR-LIN-01 (End-to-End Lineage Graph): The system shall provide an interactive Lineage View for any registered model version, detailing:
  - \* Source Experiment & Run ID
  - \* Dataset Configuration & Split Version
  - \* Hyperparameter Profile & Fixed Seed
  - \* Code Version: Git Commit Hash
  - \* Environment Provenance: Python version, PyTorch version, torchvision version, NumPy version, OS platform.

## 12.13 Model Promotion

- \* FR-PRM-01: Users shall be able to promote a registered model version to Promoted for Inference status, designating it as the active model for serving predictions.

## 12.14 Inference / Serving

- \* FR-INF-01: The system shall provide an interactive Inference Interface connected directly to the Model Registry.
- \* FR-INF-02: Users can select a promoted model version, input or upload a test image (e.g., CIFAR-10 32x32 sample), and view predicted top-k class labels, confidence percentages, inference latency (ms), and model metadata.

## 12.15 Search & Discovery

- \* FR-SRCH-01: The system shall provide global search and multi-parameter filtering across Experiments, Runs, and Models by Dataset, Architecture, Decision Status, Seed, Tag, and Test Accuracy range.

## 12.16 Decision Reporting

- \* FR-RPT-01: The system shall automatically generate a Model Selection Decision Report summarizing investigation objectives, tried approaches, strengths, limitations, rejected/superseded candidates with rationales, selected model version, and supporting reproducibility evidence.

Section takeaway: Functional requirements span 16 distinct modules, providing rigorous specification for every platform capability.

## 13. User Stories and Acceptance Criteria

### US-01: Configure and Launch Training Run

- \* User Story: As an ML Engineer, I want to configure hyperparameters and launch a PyTorch training run via a web form so that I can execute experiments without modifying script code.
- \* Acceptance Criteria:
  1. Form validates inputs (Learning Rate  $> 0$ , Epochs  $\geq 1$ , Seed integer).
  2. Submitting form creates a Run in QUEUED state, triggers background execution, and transitions state to RUNNING.
  3. UI displays toast notification and redirects to Live Run Detail view.
  4. Per-epoch metrics begin populating within 30 seconds of launch.

### US-02: Monitor Live Metric Trajectories

- \* User Story: As an ML Engineer, I want to observe live loss and accuracy charts as epochs complete so that I can monitor training convergence.
- \* Acceptance Criteria:
  1. Line charts update dynamically per epoch without full-page refreshes.
  2. Progress bar indicates current epoch vs total epochs (e.g., Epoch 12 / 30).
  3. Elapsed time and estimated time remaining update per epoch.

### US-03: Evaluate Candidate Models & Record Decision Rationale

- \* User Story: As an ML Lead, I want to compare alternative candidate runs and record why a model was selected or rejected so that our team maintains decision traceability.
- \* Acceptance Criteria:
  1. Comparison view overlays up to 4 run curves and displays a parameter summary table.
  2. User can assign candidate status (Candidate, Accepted, Rejected, Superseded, Selected).
  3. User must enter a non-empty rationale string to submit a decision.
  4. Saved decision appears immediately on the Experiment Decision Report tab.

### US-04: Conduct Reproducibility Variance Report

- \* User Story: As an AI Researcher, I want to run a multi-seed reproducibility report so that I can evaluate accuracy sensitivity to random seed variation.
- \* Acceptance Criteria:
  1. User can trigger Variance Report with N=5 controlled seeds for a chosen configuration.
  2. System executes runs sequentially, logging test accuracy for each seed.
  3. Final report displays table of run accuracies alongside Mean, Std Dev, Min, Max, and Spread %.

## US-05: Record Rejection Reasons & Link Motivating Limitations

- \* User Story: As an AI Researcher, I want to record why an approach was rejected and link a new approach to the limitation that motivated it so that future investigations do not lose the reasoning behind that decision.
- \* Acceptance Criteria:
  1. Rejection form requires selecting or entering observed limitations (e.g., insufficient accuracy, high latency).
  2. Creating a successor investigation allows selecting a parent run/limitation pointer.
  3. System displays linked successor banner (e.g., "Investigating ResNet-18 to address Custom CNN accuracy limitation").

## US-06: Trace Investigation Reasoning Chain as a Reviewer

- \* User Story: As a Reviewer / Auditor, I want to view why a selected model approach was preferred over alternatives so that I can evaluate the decision using supporting evidence.
- \* Acceptance Criteria:
  1. Reviewer opens Decision Report or Registry Lineage view for a promoted model version.
  2. View renders an unbroken chain from initial objective through rejected/superseded trials to final selection.
  3. Selecting any intermediate approach displays its strengths, limitations, rejection reasons, and why alternatives were not chosen.

Section takeaway: Acceptance criteria define unambiguous, testable conditions for all key user workflows.

## 14. Requirements Matrix

| Req ID    | Module     | Feature Summary                             | Priority  | Target Persona       | Verification Method    |
|-----------|------------|---------------------------------------------|-----------|----------------------|------------------------|
| FR-EXP-01 | Experiment | Create & manage ML investigation objectives | Must Have | ML Lead / Researcher | Manual UX / API Test   |
| FR-EXP-02 | Experiment | Store hypothesis, decisions & conclusions   | Must Have | AI Researcher        | API Schema Check       |
| FR-DAT-01 | Dataset    | Managed Dataset Catalog (CIFAR-10 baseline) | Must Have | ML Engineer          | Pipeline Execution     |
| FR-MDL-01 | Model      | Supported model architecture families       | Must Have | ML Engineer          | Model Instantiation    |
| FR-EXE-01 | Execution  | Asynchronous PyTorch training loop          | Must Have | ML Engineer          | Subprocess Integration |

|           |                 |                                               |             |                          |                          |
|-----------|-----------------|-----------------------------------------------|-------------|--------------------------|--------------------------|
| FR-EXE-03 | Execution       | Single active job handling & queueing         | Must Have   | ML Engineer              | Concurrent Launch Test   |
| FR-TRK-01 | Tracking        | Per-epoch metric logging (loss, accuracy)     | Must Have   | ML Engineer              | Metric Storage Audit     |
| FR-EVL-01 | Evaluation      | Holdout test set evaluation pass              | Must Have   | Model Evaluator          | Pipeline Verification    |
| FR-REP-01 | Reproducibility | Fixed-seed RNG initialization protocol        | Must Have   | AI Researcher            | Multi-run Determinism    |
| FR-REP-03 | Reproducibility | Automated Variance Report engine (N=5)        | Must Have   | AI Researcher            | Statistical Calculation  |
| FR-CMP-01 | Comparison      | Side-by-side candidate comparison view        | Must Have   | ML Lead                  | UX Workflow Verification |
| FR-DEC-01 | Decision        | Candidate decision state recording & evidence | Must Have   | ML Lead / Decision Maker | Audit Log Inspection     |
| FR-DEC-02 | Decision        | Motivating limitation & predecessor links     | Must Have   | AI Researcher / ML Eng   | Lineage Graph Check      |
| FR-DEC-05 | Decision        | Alternative approach rejection analysis       | Must Have   | Reviewer / Auditor       | Decision Report Audit    |
| FR-ART-01 | Artifact        | Periodic & final .pt checkpoint serialization | Must Have   | ML Engineer              | File System Check        |
| FR-REG-01 | Registry        | Model Registry with semver versioning         | Must Have   | MLOps Engineer           | Registry Service Audit   |
| FR-LIN-01 | Lineage         | Full provenance tracking (Git, Env, Dataset)  | Must Have   | MLOps / Auditor          | Lineage Graph Check      |
| FR-PRM-01 | Promotion       | Model promotion for inference serving         | Must Have   | MLOps Engineer           | Registry State Check     |
| FR-INF-01 | Inference       | FastAPI serving & prediction interface        | Must Have   | Evaluator / ML Eng       | HTTP Endpoint Test       |
| FR-RPT-01 | Report          | Automated Model Selection Decision Report     | Should Have | Decision Maker           | Report Render Check      |

*Section takeaway: The matrix maps every functional requirement to its priority, target persona, and empirical verification method.*

## 15. Non-Functional Requirements (NFRs)

### 15.1 Performance Requirements

- \* NFR-PERF-01: API endpoints for experiment listing, run details, and registry queries shall respond within < 300 ms under standard local database loads.
- \* NFR-PERF-02: Local metric logging operations per epoch shall execute within < 30 ms, introducing negligible overhead (< 1%) to training loop step time.
- \* NFR-PERF-03: Live metric chart updates in the React frontend shall render within < 500 ms of backend event generation.

### 15.2 Reliability & Fault Isolation

- \* NFR-REL-01: Training execution errors (e.g., CUDA out-of-memory, NaN loss) shall be safely trapped by the backend process manager. The run state shall update to FAILED with logged traceback without crashing the FastAPI application.
- \* NFR-REL-02: Local metric logging and checkpoint saving shall function fully offline without dependency on external cloud services.

### 15.3 Usability & Accessibility

- \* NFR-USE-01: The web dashboard shall feature a responsive design built with Tailwind CSS, optimized for desktop displays (1920x1080) and laptop viewports (1366x768).
- \* NFR-USE-02: Interface text and status badge colors shall maintain WCAG 2.1 AA compliant contrast ratios ( $\geq 4.5:1$ ).

### 15.4 Maintainability & Code Quality

- \* NFR-MNT-01: The codebase shall enforce clean separation of concerns: React frontend SPA, FastAPI REST controllers, PyTorch training orchestrator, and database persistence layers.
- \* NFR-MNT-02: Backend Python codebase shall achieve  $\geq 80\%$  line coverage across unit and integration test modules.

*Section takeaway: NFRs specify observable, testable performance thresholds, error isolation behaviors, and code quality standards.*

## 16. UX Requirements & Information Architecture

### 16.1 UX Design Philosophy

The TrainTrace user interface prioritizes clarity, data density, decision visibility, and reasoning chain transparency. It uses a modern dark-mode aesthetic with high-contrast metric visualizers, clear status tags, and structured decision cards.

### 16.2 Information Architecture & Screen Map

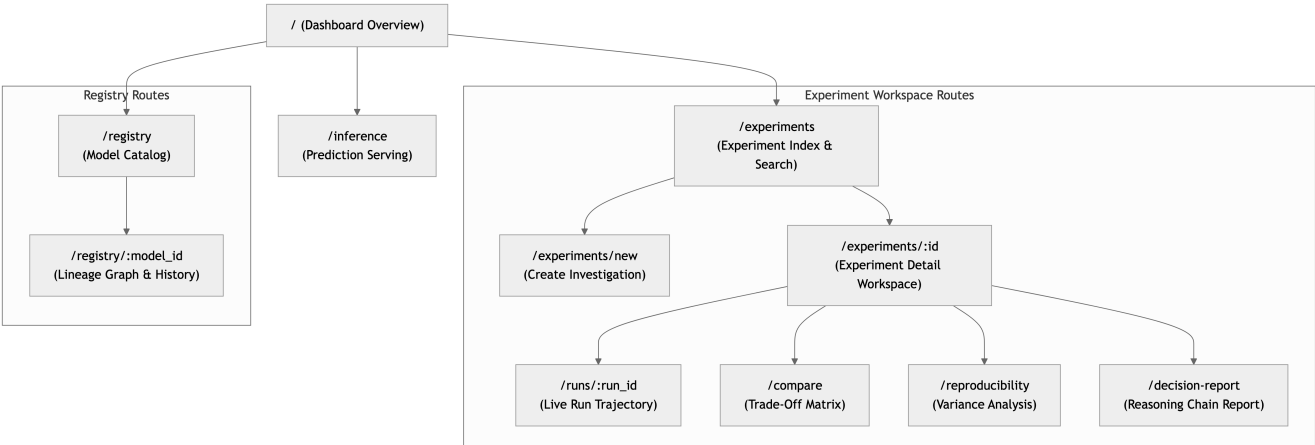


Figure 7 -- Information Architecture & Screen Map. Structural hierarchy mapping navigation routes between dashboard overview, experiment workspaces, candidate matrices, decision reports, model registries, and inference endpoints.

### 16.3 Model Candidate Lifecycle State Machine

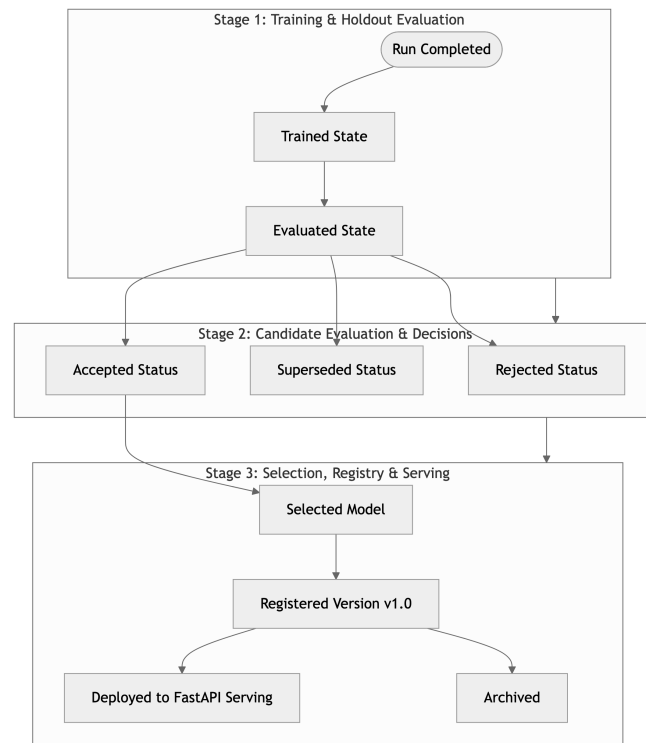


Figure 5 -- Model Candidate Lifecycle & State Transitions. State diagram mapping model candidate progression from training completion through test evaluation, decision recording, registration, promotion, and archiving.

*Section takeaway: Information architecture organizes workflow views logically, enabling rapid switching between execution details, comparison matrices, decision reports, and model registries.*

## 17. Success Metrics & Evaluation Criteria

### 17.1 Platform Evaluation Dimensions

1. Investigation Decision Rigor & Traceability: Evaluated by verifying that 100% of completed experiments contain explicitly recorded candidate decisions (Accepted/Rejected/Selected) supported by metric evidence and reasoning chains.
2. Reproducibility Verification: Evaluated by executing Protocol A (matching conditions) to confirm deterministic metric curve overlay, and Protocol B (N=5 seeds) to measure accuracy variance spread.
3. End-to-End Lineage & Reason Traceability: Evaluated by tracing a promoted model version from the Inference UI back through the Registry to its exact originating Run ID, Git commit hash, dataset split, and motivating investigation limitation.
4. Operational Full-Stack Completeness: Evaluated by executing all core use cases via the web dashboard without manual database or script intervention.

*Section takeaway: Evaluation criteria focus on empirical demonstration of reproducibility, decision traceability, and full-stack integration.*

## 18. Assumptions

---

- 1. Execution Hardware: Target hardware is a standard multi-core developer workstation (CPU-based PyTorch execution; optional CUDA GPU acceleration supported automatically if detected).
- 2. Dataset Availability: The CIFAR-10 dataset can be downloaded via standard torchvision utilities and cached locally in the filesystem.
- 3. Local Storage Infrastructure: Local SQLite or PostgreSQL database storage is available for persisting experiment metadata, metrics, and decision records.
- 4. Git Availability: The execution environment has Git installed to allow capturing current commit hashes for code provenance.

---

## 19. Constraints

---

- 1. Development Timeline: The project must be fully designed, implemented, tested, and documented within a 12-week solo developer timeframe.
- 2. Benchmark Workload Boundary: CIFAR-10 image classification serves as the primary benchmark dataset to maintain baseline consistency during evaluation.
- 3. Document Boundary: The PRD specifies product requirements (WHAT, WHY, WHO). Detailed technical code, API schemas, and database DDL belong exclusively to Technical Design Documents.

---

## 20. Dependencies

---

- 1. Core Deep Learning Framework: PyTorch 2.x, torchvision, NumPy.
- 2. Backend Runtime Engine: Python 3.10+, FastAPI, Uvicorn, Pydantic, SQLAlchemy.
- 3. Frontend Interface Framework: React 18+, Vite/Next.js, Tailwind CSS, Lucide Icons, Recharts / Chart.js.
- 4. Development & Containerization: Docker, Docker Compose, Git.

---

## 21. Risks and Mitigation Strategies

---

| Risk Description                         | Severity | Likelihood | Mitigation Strategy                                                                                                                             |
|------------------------------------------|----------|------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| Long CPU Execution Times for Deep Models | Medium   | High       | Limit default demo configurations to 20-30 epochs; use lightweight architectures (ResNet-18, Custom CNN); support adjustable batch sizes (128). |

|                                                     |        |        |                                                                                                                                                                   |
|-----------------------------------------------------|--------|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| OS-Level RNG Differences in PyTorch CPU Workers     | Low    | Medium | Enforce <code>torch.use_deterministic_algorithms(True)</code> , pin DataLoader worker seeds via <code>worker_init_fn</code> , and record OS environment metadata. |
| Failure During Training Execution (OOM / Exception) | Medium | Low    | Wrap training loop in process isolation with try-except blocks; capture tracebacks, update run status to FAILED, and preserve all metrics up to failure.          |
| Scope Creep (Enterprise Feature Over-Expansion)     | High   | Medium | Enforce strict MoSCoW prioritization; limit features to 12-week solo academic scope; defer enterprise cloud/multi-GPU features to stretch goals.                  |

Section takeaway: Proactive risk mitigations safeguard project completion within the 12-week solo constraint.

## 22. Feature Prioritization / MoSCoW

### 22.1 Must Have (Core 12-Week Deliverables)

- \* PyTorch CIFAR-10 training pipeline (DataLoaders, autograd, loss, evaluation).
- \* Fixed-seed deterministic initialization protocol.
- \* Local per-epoch metric logging and state checkpoint serialization (.pt).
- \* Experiment & Run hierarchy (Objective, Hypothesis, Runs).
- \* Experiment Investigation & Decision Traceability engine (11 attributes, reasoning chain, status recording, rejection reasons).
- \* Model Candidate Decision Engine (Accepted, Rejected, Superseded, Selected + Rationale).
- \* Model Registry with semver versioning and lineage provenance tracking.
- \* Side-by-side Candidate Comparison view with trade-off analysis tables.
- \* Multi-seed Reproducibility Variance Report engine (N=5).
- \* FastAPI model serving and interactive prediction UI.
- \* React frontend SPA + FastAPI REST API full-stack application.

### 22.2 Should Have (High-Value Enhancements)

- \* Planned Dataset Catalog (CIFAR-100, MNIST, Fashion-MNIST, SVHN).
- \* Code Git commit hash and environment metadata capture.
- \* Downloadable .pt model checkpoint files via UI.
- \* Automated Model Selection Decision Report generator.
- \* Optional Weights & Biases cloud sync integration.

## 22.3 Could Have (Stretch Extensions)

- \* MobileNetV2 and EfficientNet-B0 pre-configured model templates.
- \* Interactive Confusion Matrix render module.
- \* Learning Rate Scheduler trajectory preview.

## 22.4 Won't Have (Deferred Outside Scope)

- \* Distributed multi-GPU Distributed Data Parallel (DDP) training.
- \* Arbitrary user Python code upload execution.
- \* Automated Hyperparameter Optimization (Bayesian / Grid Search engine).
- \* Production cloud Kubernetes deployment.

Section takeaway: MoSCoW prioritization guarantees delivery of a complete, defensible platform within 12 weeks.

## 23. OJT Requirement Traceability Matrix

| Formal OJT Requirement                                            | Requirement Type          | TrainTrace PRD Coverage                                   | Traceability & Mapping                                                                            |
|-------------------------------------------------------------------|---------------------------|-----------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| PyTorch Training Pipeline<br>(DataLoaders, Autograd, Checkpoints) | Core Technical Mandate    | Section 2.3, Section 8.1, Section 12.1, 12.4, 12.10       | Fully Covered -- Core PyTorch execution loop, DataLoaders, autograd, and .pt state serialization. |
| CIFAR-10 Image Classification Dataset Benchmark                   | Core Workload Mandate     | Section 1.1, Section 5.1, Section 11.1                    | Fully Covered -- CIFAR-10 serves as mandatory baseline benchmark dataset.                         |
| Experiment Tracking Dashboard                                     | Core UI Mandate           | Section 2.3, Section 12.5, Section 13 (US-02), Section 16 | Fully Covered -- Real-time loss/accuracy chart tracking and local metric persistence.             |
| Reproducibility & Variance Analysis                               | Core Analytical Mandate   | Section 5.2, Section 12.7, Section 13 (US-04)             | Fully Covered -- Dual-mode reproducibility engine (matching seed repeatability & N=5 variance).   |
| Fixed-Seed Deterministic Protocol                                 | Core Seeding Mandate      | Section 2.3, Section 12.7 (FR-REP-01)                     | Fully Covered -- Multi-library seed initialization (PyTorch, NumPy, Python, DataLoader workers).  |
| Full-Stack Application (React Frontend + FastAPI Backend)         | Core Architecture Mandate | Section 1.1, Section 2.4, Section 16                      | Fully Covered -- Responsive React SPA communicating with FastAPI REST services.                   |
| Basic User Auth & Session Handling                                | Secondary Mandate         | Section 11.3, Section 14 (FR-AUTH-01)                     | Fully Covered -- Lightweight session authentication for multi-user dashboard access.              |

|                                |                        |                            |                                                                      |
|--------------------------------|------------------------|----------------------------|----------------------------------------------------------------------|
| Distributed Multi-GPU Training | Stretch Technical Goal | Section 11.3, Section 22.4 | Deferred -- Explicitly categorized under Stretch / Won't Have (MVP). |
|--------------------------------|------------------------|----------------------------|----------------------------------------------------------------------|

*Section takeaway: 100% of mandatory OJT requirements are explicitly mapped, preserved, and satisfied within the expanded product model.*

## 24. Known Limitations

1. Single Active Job Execution: The MVP training runner processes one active training run at a time to prevent CPU resource contention on local development hardware.
2. Dataset Catalog Boundaries: Dataset loading is restricted to the pre-configured image classification catalog (CIFAR-10, CIFAR-100, MNIST, Fashion-MNIST, SVHN). Arbitrary dataset file uploads are excluded.
3. CPU Training Duration: Training deep vision models on CPU hardware requires reduced epoch counts (20-30 epochs) for interactive demonstrations.

## 25. References

1. Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., ... & Chintala, S. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. Advances in Neural Information Processing Systems (NeurIPS 2019). arXiv:1912.01703.
2. Krizhevsky, A. (2009). Learning Multiple Layers of Features from Tiny Images. (CIFAR-10 Dataset Technical Report). University of Toronto. <https://www.cs.toronto.edu/~kriz/cifar.html>
3. PyTorch Documentation. Reproducibility & Deterministic Algorithms. <https://pytorch.org/docs/stable/notes/randomness.html>
4. FastAPI Framework Documentation. Asynchronous Server & REST API Design. <https://fastapi.tiangolo.com/>
5. React Documentation. Building Interactive Single-Page Web Applications. <https://react.dev/>
6. Weights & Biases Documentation. Experiment Tracking Best Practices. <https://docs.wandb.ai/>

End of Tracked Training Pipeline (TTP) / TrainTrace Product Requirements Document (PRD v5.2)